1. Desarrollo de software: arquitectura en capas con patrón MVC

El sistema está organizado en tres capas que corren por separado: presentación (servidor web), aplicación (servidor API) y datos (SQL Server). Son dos programas Django independientes, cada uno con su configuración y sus dependencias; el servidor web no tiene base de datos ni driver de SQL Server y obtiene todo de la API por HTTP.

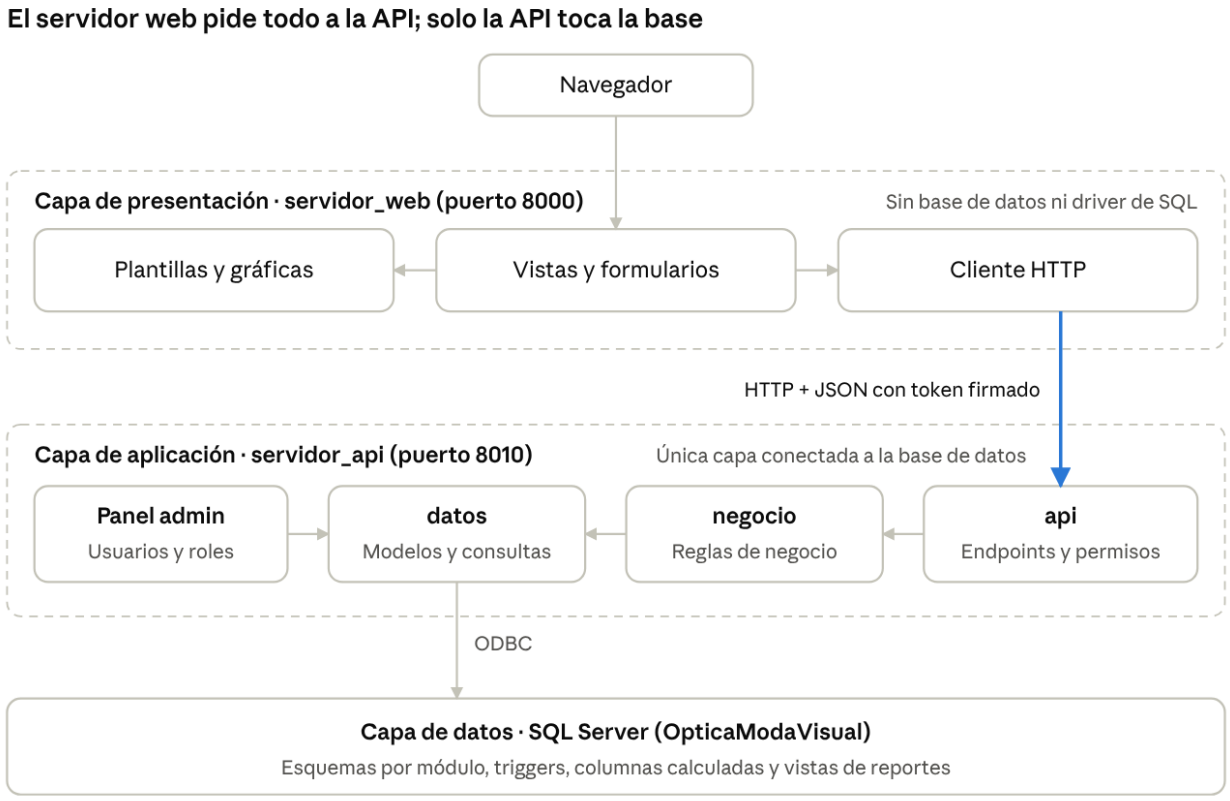


Figura 1. Arquitectura en capas: el navegador solo habla con el servidor web; el servidor web llama a la API por HTTP con un token firmado; y solo la API, a través de su capa de datos, se conecta a SQL Server.

1.1 Capas de la arquitectura

Capa	Programa	Responsabilidad	Puerto
1. Presentación	`servidor_web`	Pantallas, formularios, dashboard y exportación; consume la API.	8000
2. Aplicación	`servidor_api`	API HTTP/JSON con token, reglas de negocio y acceso a datos; panel administrativo.	8010
3. Datos	SQL Server	Esquemas por módulo, triggers, columnas calculadas, secuencias y vistas de reportes.	1433

1.2 Patrón MVC dentro de cada servidor

Django implementa MVC con el nombre MTV (Model–Template–View). Así se reparte entre los dos servidores:

Componente MVC	Servidor	Ubicación	Responsabilidad
Modelo	API	`datos/models/`, `datos/repositorios/`	Mapear las tablas de SQL Server y concentrar todas las consultas.
Lógica de negocio	API	`negocio/servicios/`	Horarios sin traslape, flujo de ventas, stock, facturación, permisos por rol.
Controlador de la API	API	`api/views.py`, `api/operaciones.py`	Validar token y permiso, ejecutar la operación, responder JSON (200/401/403/404/422).
Controlador web	Web	`presentacion/views/`, `forms/`, `urls.py`	Validar formularios, llamar a la API y elegir la pantalla.
Vista	Web	`presentacion/templates/`	HTML que ve el usuario.

Una prueba automática (`test_arquitectura.py`) lee el código fuente y falla si la presentación importa la capa de negocio o de datos, si una capa inferior importa una superior o si el servidor web tuviera base de datos.

1.3 Operaciones clave cubiertas

Módulo	Operaciones
Clientes	Alta, edición, activar/desactivar, ficha con historial (citas, exámenes, recetas, compras).
Citas	Agendar sin traslape de horario, confirmar, atender, no asistió, cancelar con motivo, cobrar la consulta.
Clínica	Examen visual (desde una cita), receta óptica, receta imprimible.
Inventario	Productos y servicios, proveedores, categorías y marcas, entradas/salidas/ajustes de stock.
Ventas	Productos y servicios con precio editable, descuento, confirmación (descuenta stock), pagos parciales, entrega, anulación (devuelve stock).
Facturación	Emisión con numeración de la BD e IVA calculado, factura imprimible, anulación.
Seguridad	Login con token, roles (Administrador, Optometrista, Vendedor), permisos por rol validados en ambos servidores, panel administrativo.
Analítica	Dashboard con indicadores y gráficas, reportes exportables.

2. Módulo analítico

La página **Reportes y analítica** ofrece un dashboard interactivo y **8 reportes exportables** a Excel (.xlsx) y CSV, por encima del mínimo de 2 que pide el entregable. Los datos salen de las vistas del esquema `reportes` de SQL Server y de consultas de la capa de datos; la API los entrega y el servidor web dibuja las gráficas y arma los archivos.

2.1 Dashboard interactivo

El periodo se elige en la misma página (últimos 7, 30 o 90 días, o último año) y todo se recalcula.

- **Indicadores:** total vendido, ticket promedio, saldo por cobrar, valor del inventario al costo, productos con stock bajo y citas del periodo.
- **Gráficas (Chart.js):** al pasar el cursor muestran el valor exacto, y al hacer clic en la leyenda se oculta o muestra cada serie.

Gráfica	Tipo	Pregunta que responde
Ventas diarias	Línea (Q) + barras (n.º de ventas), dos ejes	¿Cómo evolucionan las ventas día a día?
Ventas por categoría	Dona	¿Qué línea de producto genera más ingreso?
Productos más vendidos	Barras horizontales	¿Qué productos rotan más?
Ventas por vendedor	Barras	¿Cómo rinde cada vendedor?
Citas por estado	Pastel	¿Cuántas citas se atienden, cancelan o se pierden?

2.2 Reportes exportables

Cada reporte tiene dos botones: **Excel**, con título, encabezados con formato, montos en quetzales y fechas reales, y **CSV**, en UTF-8 compatible con Excel. La página también se puede imprimir o guardar como PDF.

#	Reporte	Origen	Usa el periodo
1	Ventas diarias	Vista `reportes.vw_ventas_diarias`	Sí
2	Ventas por vendedor	Vista `reportes.vw_ventas_por_vendedor`	No (histórico)
3	Ventas por categoría	Consulta ventas + inventario	Sí
4	Productos más vendidos	Vista `reportes.vw_productos_mas_vendidos`	No (histórico)
5	Inventario valorizado	Consulta inventario (existencias × costo y × precio)	No (al día)
6	Productos con stock bajo	Vista `reportes.vw_stock_bajo`	No (al día)
7	Saldos por cobrar	Vista `reportes.vw_saldo_ventas`	No (al día)
8	Agenda de citas	Vista `reportes.vw_agenda_citas`	Sí

3. Plan de pruebas

La calidad se verificó con cuatro tipos de prueba: unitarias por módulo, integración con la base de datos, integración de punta a punta entre las tres capas y verificación de integridad de los datos. Todo está automatizado y se ejecuta con `python manage.py test`.

Tipo	Qué verifica	Base de datos	Cantidad
Unitarias por módulo	Cada regla de negocio, el cliente HTTP, formularios, permisos, exportación y arquitectura, de forma aislada.	No: la capa de datos se reemplaza con objetos simulados (`unittest.mock`).	87
Integración con la BD	Servicios reales contra SQL Server: triggers, columnas calculadas, secuencias, vistas y la API por HTTP.	Copia de la base	12
Integración entre capas	Recorridos completos navegador → servidor web → API → SQL Server.	Copia de la base	8
Integridad de datos	15 reglas de coherencia entre esquemas sin llave foránea.	Base real (solo lectura) y copia	15 reglas

3.1 Entorno y herramientas

Windows 11, Python 3.13, Django 5.2.18, mssql-django 2.0, SQL Server 2022 (instancia `MSSQLSERVER123`), ODBC Driver 18. Herramientas: el framework de pruebas de Django (`unittest`), `unittest.mock`, el cliente de pruebas de Django y `openpyxl` para validar los archivos de Excel.

3.2 Protección de los datos reales

Las pruebas con base de datos corren sobre `OpticaModaVisual_pruebas`, una copia exacta creada con `python manage.py preparar_bd_pruebas` (respaldo `COPY_ONLY`). El ejecutor de pruebas apunta siempre a esa copia, nunca la recrea ni la borra, y se detiene si no existe. Cada prueba de integración corre dentro de una transacción que se revierte, excepto las que provocan errores de triggers, que limpian lo que crean.

3.3 Criterios de aceptación

- 100 % de las pruebas aprobadas.
- 0 registros inconsistentes en las 15 reglas de integridad.
- Ningún import que cruce capas.

4. Pruebas unitarias por módulo

Las **87 pruebas unitarias pasaron (87 de 87)**. Cada servicio se prueba aislado: los repositorios se sustituyen por objetos simulados, así que la prueba falla solo si falla la regla, no la base de datos.

#	Módulo	Qué se verifica	Pruebas	Resultado
1	Seguridad y permisos	Mensajes de triggers llegan al usuario; secciones por rol (Optometrista solo clínica).	8	8/8
2	Citas	Fecha futura, horario sin traslape, paciente activo, cálculo de hora fin, transiciones de estado, motivo de cancelación.	8	8/8
3	Ventas	Precio de lista vs. precio editable de servicios, stock, descuentos, confirmación, pagos parciales y totales, anulación, cobro de citas.	20	20/20
4	Facturación	Prorrateo del descuento al céntimo, una factura por venta, estados facturables, anulación.	6	6/6
5	Inventario	Código único, precio ≥ costo, movimientos inválidos, ajuste de stock por diferencia.	4	4/4
6	Clientes	DPI único, cliente nuevo activo.	2	2/2
7	Panel y reportes	Porcentajes de la gráfica semanal, periodos válidos, columnas de exportación, mínimo 2 reportes.	4	4/4
8	API	Serialización de fechas y decimales, token firmado, catálogo de operaciones, códigos HTTP 400/401/403/404/422/200.	13	13/13
9	Arquitectura	Ninguna capa importa otra que no le corresponde; el servidor web no tiene base de datos.	4	4/4
10	Cliente HTTP (web)	Codificación de datos, token en cada petición, traducción de errores, API caída.	6	6/6
11	Presentación (web)	Validaciones de formularios, permisos del menú, sesión, Excel y CSV.	12	12/12
	Total		**87**	**87/87**

Los módulos 1 a 9 están en `servidor_api/pruebas/unitarias/` y los 10 y 11 en `servidor_web/pruebas/unitarias/`.

4.1 Casos de prueba representativos

ID	Caso	Entrada	Resultado esperado	Obtenido
CP-01	Cita en el pasado	Fecha de ayer	Error «Selecciona una fecha y hora futuras»; no se guarda.	Aprobado
CP-02	Horario ocupado	Otra cita vigente se traslapa	Error «Ese horario ya está ocupado».	Aprobado
CP-03	Cita atendida no se cancela	Estado ATENDIDA → CANCELADA	Error de transición inválida.	Aprobado
CP-04	Precio editable solo en servicios	Producto con precio distinto al de lista	Error «Solo los servicios permiten cambiar el precio».	Aprobado
CP-05	Consulta con precio especial	Servicio LASIK a Q 5,800	La línea se guarda a Q 5,800.	Aprobado
CP-06	Stock insuficiente	2 unidades con 1 en existencia	Error «Stock insuficiente: hay 1 disponibles».	Aprobado
CP-07	Confirmar descuenta solo productos	Venta con 1 producto y 1 servicio	Un solo movimiento SALIDA (el producto) y estado CONFIRMADA.	Aprobado
CP-08	Pago completo	Q 300 + Q 350 sobre Q 650	Primer pago parcial; el segundo deja la venta PAGADA.	Aprobado
CP-09	Anular con factura vigente	Venta con factura A-8 emitida	Error «Primero anula la factura A-8».	Aprobado
CP-10	Prorrateo del descuento	Q 25 sobre líneas de Q 500, 150 y 100	Partes que suman exactamente Q 25.00.	Aprobado
CP-11	Optometrista	Rol OPTOMETRISTA	Solo Inicio, Clientes, Citas y Clínica.	Aprobado
CP-12	Operación no publicada en la API	`ventas._exigir_estado`	No se puede invocar (404).	Aprobado
CP-13	Regla de negocio por HTTP	Servicio lanza «DPI duplicado»	Respuesta 422 con mensaje y campo `dpi`.	Aprobado
CP-14	Capas	Revisión de imports de todo el código	La presentación no importa negocio, datos ni drivers de BD.	Aprobado

5. Pruebas de integración y verificación de integración de datos

Las **20 pruebas de integración pasaron (20 de 20)** y las **15 reglas de integridad** no encontraron ningún registro inconsistente, ni en la base real ni en la copia de pruebas.

5.1 Integración con la base de datos (12 pruebas)

Archivo `servidor_api/pruebas/integracion/test_base_de_datos.py`, contra SQL Server real (copia), con los servicios y repositorios de producción.

ID	Prueba	Qué confirma de la base	Resultado
PI-01	Integridad de datos	Las 15 reglas de la sección 5.3 devuelven 0 filas.	Aprobado
PI-02	Stock inicial	El stock inicial queda como movimiento ENTRADA (trigger `trg_movimiento_ai`).	Aprobado
PI-03	Ajuste desde el panel	Pasar de 10 a 4 unidades registra un AJUSTE de −6.	Aprobado
PI-04	Stock negativo	El trigger rechaza la salida y su mensaje «Stock insuficiente…» llega al usuario.	Aprobado
PI-05	Venta anulada	El trigger `trg_venta_au` impide reactivarla.	Aprobado
PI-06	Venta vacía	El trigger impide confirmar una venta sin productos.	Aprobado
PI-07	Venta completa	Subtotal por trigger (Q 620), total calculado (Q 600), stock 5→3, evento VentaConfirmada, pagos, factura con IVA de la BD (Q 64.29), anulación que devuelve el stock y publica VentaAnulada.	Aprobado
PI-08	Flujo clínico	Cita sin traslape → atendida → examen → receta; examen y receta no se duplican.	Aprobado
PI-09	Reportes	El tablero y los 8 reportes leen las vistas `reportes.*` con columnas completas.	Aprobado
PI-10	API por HTTP	Login → token → consulta de clientes por HTTP.	Aprobado
PI-11	Permisos en la API	El optometrista recibe 403 en Ventas y 200 en Clínica.	Aprobado
PI-12	Credenciales inválidas	El login responde 401.	Aprobado

5.2 Integración entre capas (8 pruebas)

Archivo `servidor_web/pruebas/test_integracion_niveles.py`: un navegador simulado usa el servidor web, que llama por HTTP a la API, que trabaja sobre SQL Server.

ID	Prueba	Recorrido	Resultado
PN-01	Navegación	Login inválido rechazado; las 15 pantallas principales responden.	Aprobado

ID	Prueba	Recorrido	Resultado
PN-02	Flujo completo	Cliente → producto (stock 5) → cita confirmada y atendida → examen → receta → cobro de la cita → producto agregado (stock 3 al confirmar) → pago de Q 650 → factura con IVA Q 69.64 → entrega.	Aprobado
PN-03	Reglas de negocio	Los errores de la API (fecha pasada, código duplicado) aparecen como mensajes en pantalla.	Aprobado
PN-04	Registro inexistente	Una venta que no existe muestra la página 404.	Aprobado
PN-05	Analítica	El dashboard carga sus datos; Excel (.xlsx válido) y CSV se descargan; un reporte inexistente da 404.	Aprobado
PN-06	Optometrista	Bloqueado en Ventas en el servidor web y también en la API con su propio token (403).	Aprobado
PN-07	API sin token	Responde 401.	Aprobado
PN-08	API caída	El servidor web muestra la página «No hay conexión con el servidor de aplicación» (503).	Aprobado

5.3 Verificación de integridad de datos (15 reglas)

Varias relaciones entre esquemas no tienen llave foránea (por ejemplo, `ventas.venta.id_cliente`), así que se verifican con consultas. Comando: `python manage.py verificar_integridad` (solo lectura).

#	Regla	Base real	Copia
1	El stock de cada producto es igual a la suma de sus movimientos.	OK	OK
2	Los servicios no tienen existencias ni movimientos.	OK	OK
3	El subtotal de cada venta es la suma de sus líneas.	OK	OK
4	Toda venta pertenece a un cliente existente.	OK	OK
5	Toda línea de venta apunta a un producto existente.	OK	OK
6	Las ventas PAGADAS o ENTREGADAS están cubiertas por sus pagos.	OK	OK
7	Ninguna venta tiene pagos por más de su total.	OK	OK
8	Cada producto de una venta confirmada tiene su movimiento de SALIDA.	OK	OK
9	El total de cada factura emitida es igual al de su venta.	OK	OK
10	Las líneas de cada factura suman su total.	OK	OK
11	Toda factura corresponde a una venta existente.	OK	OK
12	Toda cita pertenece a un cliente existente.	OK	OK
13	Todo examen pertenece a un cliente existente.	OK	OK
14	La cita de un examen es del mismo paciente.	OK	OK
15	Todo usuario del sistema tiene un rol existente.	OK	OK

6. Conclusiones, defectos y limitaciones

El prototipo cumple el Entregable 3: las operaciones clave funcionan sobre una arquitectura en capas con MVC, el módulo analítico ofrece 5 gráficas interactivas y 8 reportes exportables, y las 107 pruebas automatizadas más las 15 reglas de integridad pasan sin fallas. Las pruebas encontraron defectos reales, todos corregidos antes de la corrida oficial.

6.1 Defectos encontrados y corregidos

#	Defecto	Cómo se detectó	Corrección
1	La presentación importaba modelos de la capa de datos en 4 archivos (formularios y admin).	Revisión de dependencias	Formularios sin modelos; el admin pasó al servidor de aplicación; prueba `test_arquitectura.py`.
2	Las llaves IDENTITY estaban declaradas como enteros normales e impedían insertar ventas, facturas y proveedores.	Integración con la BD	Declaradas como `AutoField`.
3	Django intentaba escribir columnas calculadas por SQL Server (total, IVA).	Integración con la BD	Declaradas como `GeneratedField`.
4	Las fechas se guardaban en UTC y las vistas de reportes en hora local: las ventas de la noche caían al día siguiente.	Integración con la BD	Hora de Guatemala en toda la aplicación.
5	Un THROW en un trigger revierte la transacción completa en SQL Server.	Integración con la BD	Cada operación en su propia transacción; esas pruebas corren sin transacción envolvente.
6	Un doble clic en Guardar del panel administrativo provocaba un error de llave duplicada.	Reproducido con dos envíos simultáneos	Botón deshabilitado tras el primer clic y aviso en lugar de error.
7	El puerto 8001 estaba ocupado por Cisco Packet Tracer en el equipo de desarrollo.	Arranque del servidor API	La API usa el puerto 8010 (configurable).

6.2 Limitaciones conocidas

- La comunicación entre servidores usa HTTP sin cifrar; en producción debe ir por HTTPS y con claves secretas propias en variables de entorno.
- Las gráficas cargan Chart.js desde internet (cdnjs); sin conexión, el dashboard muestra los indicadores y tablas, pero no las gráficas.
- Los eventos que la base publica en `ventas.outbox_evento` (VentaConfirmada, VentaAnulada) se registran, pero ningún proceso los consume todavía.
- Las pruebas de integración requieren una instancia de SQL Server con permiso para respaldar y restaurar (para crear la copia).